

ASSIGNMENT-2

Course: Full Stack Web Development

Course Code: 23CM4121

Branch: CSM

Section: B

Task No.: Assignment 2

Student Name: K. Saranya

Roll No.: A24126552090

1. TITLE

Build a Student Record Management System using MongoDB

2. AIM

To develop a simple Student Record Management System using MongoDB that can store, retrieve, update, and manage student information through CRUD operations, filtering, sorting, and indexing.

3. OBJECTIVE

The objectives of this assignment are:

- To create a MongoDB database (collegeDB) and a collection (students) inside it.
- To insert multiple student documents into the collection using insertMany().
- To display all student records using find().
- To filter and display students belonging to a particular branch.
- To filter and display students who scored more than a given number of marks.
- To search for a single student using their rollNo.
- To search students using a different condition, such as their year of study.
- To update one field (marks) and a different field (email) of a specific student using updateOne().
- To delete a student record using rollNo with deleteOne().
- To display students sorted in descending order of marks.
- To create an index on rollNo and understand why indexing speeds up searches.
- To run further real-time queries: students above 80 marks, below 50 marks, the highest scorer, students of a given branch, and students sorted by marks.

4. THEORY

MongoDB is a NoSQL, document-oriented database. Instead of rows and columns, it stores data as documents — JSON-like records (technically BSON) — grouped into collections inside a database. A

collection does not enforce a fixed schema, so documents in the same collection can have different fields if needed.

`use("collegeDB")` switches the shell's context to the collegeDB database, creating it the first time data is actually written to it. `db.createCollection("students")` explicitly creates the students collection inside it.

CRUD operations in MongoDB map onto four kinds of methods:

- **Create** — `insertOne()` / `insertMany()` add one or more documents to a collection.
- **Read** — `find()` retrieves documents, optionally filtered by a query object, sorted with `.sort()`, and limited with `.limit()`.
- **Update** — `updateOne()` / `updateMany()` modify fields of matching documents, typically using the `$set` operator to change specific fields without touching the rest of the document.
- **Delete** — `deleteOne()` / `deleteMany()` remove documents that match a filter.

Query operators such as `$gt` (greater than) and `$lt` (less than) let a filter compare a field's value instead of only matching it exactly, e.g. `{ marks: { $gt: 75 } }` matches every document whose marks field is above 75.

Indexing: without an index, MongoDB must perform a collection scan (COLLSCAN) — reading every single document to check whether it matches a query — which gets slower as the collection grows. An index, created with `createIndex()`, builds a separate, sorted data structure (a B-tree) on the indexed field. A query that filters or sorts on an indexed field can then use an index scan (IXSCAN) to jump almost directly to the matching documents instead of reading the whole collection, which is why `db.students.createIndex({ rollNo: 1 })` makes searching by rollNo far faster once the students collection grows large — the difference between checking every document and looking the value up directly.

5. HOW TO BUILD THIS PROJECT — STEP-BY-STEP SETUP GUIDE

Install MongoDB Community Server

Download and install MongoDB Community Server for your OS from [mongodb.com](https://www.mongodb.com). On Windows/macOS the installer also offers to install MongoDB Compass (a GUI) alongside it.

Install the MongoDB Shell (mongosh)

`mongosh` is the modern command-line shell used to run JavaScript-style queries against MongoDB. It is bundled with recent installers, or can be installed separately from the MongoDB Download Center.

Start the MongoDB server (mongod)

Run the `mongod` process so a database server is listening (by default on port 27017). On Windows this usually runs as a service automatically after installation; on macOS/Linux it may need to be started manually, e.g. `mongod --dbpath /path/to/data`.

Connect using mongosh

Open a terminal and run `mongosh`. This connects to the local MongoDB server and opens an interactive JavaScript-style shell where database commands can be typed directly.

Switch to (create) the database

Run `use("collegeDB")` (or use `collegeDB` in older shell syntax). MongoDB does not create the database file on disk until the first document is actually inserted into it.

Create the students collection

Run `db.createCollection("students")` to explicitly create the collection. (This step is optional in practice — inserting a document into a non-existent collection creates it automatically — but doing it explicitly here documents the step clearly.)

Insert sample student documents

Run `db.students.insertMany([...])` with an array of student objects to populate the collection with starting data.

Run read, update, and delete queries

Use `find()`, `updateOne()`, and `deleteOne()` with the filters and operators described in this report to retrieve, modify, and remove records.

Create an index and verify it

Run `db.students.createIndex({ rollNo: 1 })` and then `db.students.getIndexes()` to confirm the index now exists alongside the default `_id` index.

(Optional) Save the script and re-run it

Save all the commands in a .js file (e.g. `mongo_script.js`) and run the whole file at once from a terminal with `mongosh collegeDB mongo_script.js`, instead of typing each command interactively.

(Optional) Inspect the data visually

Open MongoDB Compass and connect to `localhost:27017` to browse the `collegeDB` database and the `students` collection in a table/document view, which is often easier for spot-checking data than reading raw shell output.

6. ALGORITHM / PROCEDURE

- Switch to the `collegeDB` database with `use("collegeDB")` and create the `students` collection.
- Insert four student documents in one call using `insertMany()`, each with `rollNo`, `name`, `branch`, `year`, `marks`, and `email`.
- Retrieve and display every document in the collection using `find()`.
- Filter and display only the documents where `branch` equals `"CSE-AIML"`.
- Filter and display only the documents where `marks` is greater than 75, using the `$gt` operator.
- Search for and display the single document whose `rollNo` equals `"23CM001"`.
- Filter and display documents using a different field as the condition — here, `year` equal to 3.
- Update the `marks` field of the student with `rollNo` `"23CM001"` to 90 using `updateOne()` with `$set`.
- Update the `email` field of the same student to `"ravi.kumar@example.com"` using a second `updateOne()` call with `$set`.

- Retrieve all documents sorted by marks in descending order using `.sort({ marks: -1 })`.
- Create an ascending index on the `rollNo` field using `createIndex()`, then list all indexes with `getIndexes()`.
- Run five real-time queries: marks above 80, marks below 50, the single highest-scoring student (`sort + limit(1)`), students in the CSE branch, and all students sorted by marks.
- Delete the document whose `rollNo` equals "23CM004" using `deleteOne()`, then display the remaining students using `find()`.

7. PROGRAM

mongo_script.js (run with `mongosh`)

```
// Select database
use("collegeDB");

// Create collection
db.createCollection("students");

// Insert students
db.students.insertMany([
  {
    rollNo: "23CM001",
    name: "Ravi Kumar",
    branch: "CSE-AIML",
    year: 3,
    marks: 85,
    email: "ravi@example.com"
  },
  {
    rollNo: "23CM002",
    name: "Priya Sharma",
    branch: "CSE-AIML",
    year: 3,
    marks: 92,
    email: "priya@example.com"
  },
  {
    rollNo: "23CM003",
    name: "Arjun Reddy",
    branch: "CSE",
    year: 2,
    marks: 68,
    email: "arjun@example.com"
  },
  {
    rollNo: "23CM004",
    name: "Sneha Rao",
    branch: "ECE",
    year: 3,
    marks: 45,
    email: "sneha@example.com"
  }
])
```

```

    }
  });

  // 1. Display all students
  db.students.find();

  // 2. Display students belonging to CSE-AIML
  db.students.find({ branch: "CSE-AIML" });

  // 3. Display students who scored more than 75 marks
  db.students.find({ marks: { $gt: 75 } });

  // 4. Search student using rollNo
  db.students.find({ rollNo: "23CM001" });

  // 5. Search students based on year
  db.students.find({ year: 3 });

  // 6. Update marks of a particular student
  db.students.updateOne(
    { rollNo: "23CM001" },
    { $set: { marks: 90 } }
  );

  // 7. Update another field - email
  db.students.updateOne(
    { rollNo: "23CM001" },
    { $set: { email: "ravi.kumar@example.com" } }
  );

  // 8. Display students in descending order of marks
  db.students.find().sort({ marks: -1 });

  // 9. Create index on rollNo
  db.students.createIndex({ rollNo: 1 });

  // 10. Display indexes
  db.students.getIndexes();

  // 11. Real-Time Query - Students scoring above 80
  db.students.find({ marks: { $gt: 80 } });

  // 12. Real-Time Query - Students scoring below 50
  db.students.find({ marks: { $lt: 50 } });

  // 13. Real-Time Query - Highest-scoring student
  db.students.find().sort({ marks: -1 }).limit(1);

  // 14. Real-Time Query - Students belonging to CSE
  db.students.find({ branch: "CSE" });

  // 15. Real-Time Query - Students sorted according to marks
  db.students.find().sort({ marks: -1 });

  // 16. Delete student using rollNo
  db.students.deleteOne({ rollNo: "23CM004" });

```

```
// 17. Display remaining students after deletion
db.students.find();
```

8. CODE EXPLANATION

Code	Explanation
<code>use("collegeDB")</code>	Switches the shell to the collegeDB database, creating it on first write.
<code>db.createCollection("students")</code>	Explicitly creates the students collection inside the current database.
<code>db.students.insertMany([...])</code>	Inserts an array of four student documents into the collection in one call.
<code>db.students.find()</code>	Returns every document in the collection.
<code>find({ branch: "CSE-AIML" })</code>	Returns only documents whose branch field exactly matches "CSE-AIML".
<code>find({ marks: { \$gt: 75 } })</code>	Returns documents whose marks field is strictly greater than 75.
<code>find({ rollNo: "23CM001" })</code>	Returns the document(s) whose rollNo exactly matches the given value.
<code>find({ year: 3 })</code>	Searches using a different field (year) as the filter condition.
<code>updateOne(filter, { \$set: {...} })</code>	Finds the first document matching filter and sets the given field(s) to new values (used for marks and email).
<code>find().sort({ marks: -1 })</code>	Returns all documents ordered by marks from highest to lowest (-1 = descending).
<code>createIndex({ rollNo: 1 })</code>	Builds an ascending index on rollNo so lookups on that field no longer need a full collection scan.
<code>getIndexes()</code>	Lists every index that currently exists on the collection.
<code>find({ marks: { \$lt: 50 } })</code>	Returns documents whose marks field is strictly less than 50.
<code>.limit(1)</code>	Restricts the result to only the first document of the (sorted) result set.
<code>deleteOne({ rollNo: "23CM004" })</code>	Deletes the first document matching rollNo "23CM004", if one exists.

9. TEST CASES AND EXECUTION DATA

Each command was executed against the collegeDB.students collection and the result was recorded below.

Test Case	Action	Expected Result	Actual Result	Status
TC-01	<code>find()</code> — display all students	All inserted documents are returned	Student documents returned, matching the inserted data	Pass
TC-02	<code>find({ branch: "CSE-AIML" })</code>	Only CSE-AIML students (Ravi, Priya) are returned	Ravi Kumar and Priya Sharma returned (repeated entries from earlier script runs also listed)	Pass
TC-03	<code>find({ marks: { \$gt: 75 } })</code>	Only students scoring above 75 are returned	Ravi (marks 92/90) and Priya (92) returned	Pass

Test Case	Action	Expected Result	Actual Result	Status
TC-04	find({ rollNo: "23CM001" })	Only Ravi Kumar's document(s) are returned	Documents with rollNo 23CM001 returned	Pass
TC-05	find({ year: 3 }) — condition-based search	Students in year 3 are returned	Ravi, Priya, and Sneha (year 3) returned	Pass
TC-06	updateOne() — set marks to 90 for 23CM001	The matching document's marks field is updated	matchedCount: 1, modifiedCount: 1	Pass
TC-07	updateOne() — set email for 23CM001	The matching document's email field is updated	matchedCount: 1, modifiedCount: 0 — email was already ravi.kumar@example.com, so nothing changed	Pass (no-op)
TC-08	createIndex({ rollNo: 1 }) and getIndexes()	New ascending index on rollNo is created and listed with the default _id index	Index name "rollNo_1" returned; 2 indexes listed: _id_ and rollNo_1	Pass
TC-09	find({ marks: { \$lt: 50 } })	Only students scoring below 50 are returned	Sneha Rao (marks 45) returned	Pass

10. OUTPUT

Output 1: Display all students — find()

```
collegeDB> db.students.find()
[
  {
    _id: ObjectId('6ac0a64d1893fd70e9abc114'),
    rollNo: '23CM001',
    name: 'Ravi Kumar',
    branch: 'CSE-AIML',
    year: 3,
    marks: 92,
    email: 'ravi.kumar@example.com'
  },
  {
    _id: ObjectId('6ac0a64d1893fd70e9abc115'),
    rollNo: '23CM002',
    name: 'Priya Sharma',
    branch: 'CSE-AIML',
    year: 3,
    marks: 92,
    email: 'priya@example.com'
  },
  {
    _id: ObjectId('6ac0a64d1893fd70e9abc116'),
    rollNo: '23CM003',
    name: 'Arjun Reddy',
    branch: 'CSE',
    year: 2,
    marks: 68,
    email: 'arjun@example.com'
  },
  {
    _id: ObjectId('6ac0a64d1893fd70e9abc117'),
    rollNo: '23CM004',
    name: 'Sneha Rao',
    branch: 'ECE',
    year: 3,
    marks: 45,
    email: 'sneha@example.com'
  }
]
```


Output 2: Students in branch CSE-AIML

```
collegeDB> db.students.find({ branch: "CSE-AIML" })
[
  {
    _id: ObjectId('6ac0a64d1893fd70e9abc114'),
    rollNo: '23CM001',
    name: 'Ravi Kumar',
    branch: 'CSE-AIML',
    year: 3,
    marks: 92,
    email: 'ravi.kumar@example.com'
  },
  {
    _id: ObjectId('6ac0a64d1893fd70e9abc115'),
    rollNo: '23CM002',
    name: 'Priya Sharma',
    branch: 'CSE-AIML',
    year: 3,
    marks: 92,
    email: 'priya@example.com'
  },
  {
    _id: ObjectId('6ac0a66f63253e64c8abc114'),
    rollNo: '23CM001',
    name: 'Ravi Kumar',
    branch: 'CSE-AIML',
    year: 3,
    marks: 90,
    email: 'ravi@example.com'
  },
  {
    _id: ObjectId('6ac0a66f63253e64c8abc115'),
    rollNo: '23CM002',
    name: 'Priya Sharma',
    branch: 'CSE-AIML',
    year: 3,
    marks: 92,
    email: 'priya@example.com'
  }
]
```

Output 3: Students scoring more than 75

```
collegeDB> db.students.find({ marks: { $gt: 75 } })
[
  {
    _id: ObjectId('6ac0a64d1893fd70e9abc114'),
    rollNo: '23CM001',
    name: 'Ravi Kumar',
    branch: 'CSE-AIML',
    year: 3,
    marks: 92,
    email: 'ravi.kumar@example.com'
  },
  {
    _id: ObjectId('6ac0a64d1893fd70e9abc115'),
    rollNo: '23CM002',
    name: 'Priya Sharma',
    branch: 'CSE-AIML',
    year: 3,
    marks: 92,
    email: 'priya@example.com'
  },
  {
    _id: ObjectId('6ac0a66f63253e64c8abc114'),
    rollNo: '23CM001',
    name: 'Ravi Kumar',
    branch: 'CSE-AIML',
    year: 3,
    marks: 90,
    email: 'ravi@example.com'
  },
  {
    _id: ObjectId('6ac0a66f63253e64c8abc115'),
    rollNo: '23CM002',
    name: 'Priya Sharma',
    branch: 'CSE-AIML',
    year: 3,
    marks: 92,
  }
]
```

Output 4: Search student by rollNo

```
collegeDB> db.students.find({ rollNo: "23CM001" })
[
  {
    _id: ObjectId('6ac0a64d1893fd70e9abc114'),
    rollNo: '23CM001',
    name: 'Ravi Kumar',
    branch: 'CSE-AIML',
    year: 3,
    marks: 92,
    email: 'ravi.kumar@example.com'
  },
  {
    _id: ObjectId('6ac0a66f63253e64c8abc114'),
    rollNo: '23CM001',
    name: 'Ravi Kumar',
    branch: 'CSE-AIML',
    year: 3,
    marks: 90,
    email: 'ravi@example.com'
  },
  {
    _id: ObjectId('6ac0a6ccaae5a999d1abc114'),
    rollNo: '23CM001',
    name: 'Ravi Kumar',
    branch: 'CSE-AIML',
    year: 3,
    marks: 90,
    email: 'ravi@example.com'
  },
  {
    _id: ObjectId('6ac0ac99aab4c22da9abc114'),
    rollNo: '23CM001',
    name: 'Ravi Kumar',
    branch: 'CSE-AIML',
    year: 3,
    marks: 85,
    email: 'ravi@example.com'
  }
]
```

Output 5: Students in year 3 (condition-based search)

```
collegeDB> db.students.find({ year: 3 })
[
  {
    _id: ObjectId('6ac0a64d1893fd70e9abc114'),
    rollNo: '23CM001',
    name: 'Ravi Kumar',
    branch: 'CSE-AIML',
    year: 3,
    marks: 92,
    email: 'ravi.kumar@example.com'
  },
  {
    _id: ObjectId('6ac0a64d1893fd70e9abc115'),
    rollNo: '23CM002',
    name: 'Priya Sharma',
    branch: 'CSE-AIML',
    year: 3,
    marks: 92,
    email: 'priya@example.com'
  },
  {
    _id: ObjectId('6ac0a64d1893fd70e9abc117'),
    rollNo: '23CM004',
    name: 'Sneha Rao',
    branch: 'ECE',
    year: 3,
    marks: 45,
    email: 'sneha@example.com'
  },
  {
    _id: ObjectId('6ac0a66f63253e64c8abc114'),
    rollNo: '23CM001',
    name: 'Ravi Kumar',
    branch: 'CSE-AIML',
    year: 3,
    marks: 90,
```

Output 6: Update marks for rollNo 23CM001

```
collegeDB> db.students.updateOne(
|   { rollNo: "23CM001" },
|   { $set: { marks: 90 } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
collegeDB> █
```

Output 7: Update email for rollNo 23CM001

```
collegeDB> db.students.updateOne(
|   { rollNo: "23CM001" },
|   { $set: { email: "ravi.kumar@example.com" } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 0,
  upsertedCount: 0
}
collegeDB> █
```

Output 8: Create index on rollNo and list all indexes

```
collegeDB> db.students.createIndex({ rollNo: 1 })
rollNo_1
collegeDB> db.students.getIndexes()
[
  {
    v: 2,
    key: { _id: 1 },
    name: '_id_',
    collation: { locale: 'simple' }
  },
  {
    v: 2,
    key: { rollNo: 1 },
    name: 'rollNo_1',
    collation: { locale: 'simple' }
  }
]
collegeDB> █
```

Output 9 (Real-Time): Students scoring above 80 / below 50

```
collegeDB> db.students.find({ marks: { $gt: 80 } }); db.students.find({ marks: { $lt: 50 } })
[
  {
    _id: ObjectId('6ac0a64d1893fd70e9abc117'),
    rollNo: '23CM004',
    name: 'Sneha Rao',
    branch: 'ECE',
    year: 3,
    marks: 45,
    email: 'sneha@example.com'
  },
  {
    _id: ObjectId('6ac0a66f63253e64c8abc117'),
    rollNo: '23CM004',
    name: 'Sneha Rao',
    branch: 'ECE',
    year: 3,
    marks: 45,
    email: 'sneha@example.com'
  },
  {
    _id: ObjectId('6ac0a6ccaae5a999d1abc117'),
    rollNo: '23CM004',
    name: 'Sneha Rao',
    branch: 'ECE',
    year: 3,
    marks: 45,
    email: 'sneha@example.com'
  },
  {
    _id: ObjectId('6ac0ac99aab4c22da9abc117'),
    rollNo: '23CM004',
    name: 'Sneha Rao',
    branch: 'ECE',
    year: 3,
    marks: 45,
```

11. RESULT

Thus, a simple Student Record Management System was successfully built using MongoDB. The students collection was created and populated, queried by branch/marks/rollNo/year, updated, sorted, and indexed on rollNo — covering the CRUD and query requirements, including the real-time extension queries.